

CoreNN 文件索引与搜索系统

技术报告

王家齐

January 20, 2026

Abstract

本报告详细介绍了基于 CoreNN 向量数据库的大规模文件索引与搜索系统的设计、实现与性能分析。该系统能够对近千万个文件进行高效索引，并提供毫秒级的搜索响应。系统采用 Rust 实现核心向量数据库，使用 Python 构建索引器和 Web 服务，实现了高性能的文件检索功能。

Contents

1	引言	2
1.1	背景	2
1.2	目标	2
2	系统架构	2
2.1	整体架构	2
2.2	核心组件	2
2.2.1	CoreNN 向量数据库	2
2.2.2	文件索引器	3
3	实现细节	3
3.1	索引构建流程	3
3.2	搜索算法	3
4	性能分析	3
4.1	索引构建性能	3
4.2	搜索性能	3
4.3	内存使用	3
5	系统使用	4
5.1	启动服务	4
5.2	API 使用示例	4
6	问题与优化	5
6.1	当前问题	5
6.1.1	搜索结果准确性	5
6.2	优化方案	5
6.2.1	改进向量表示	5
6.2.2	优化搜索算法	5

7	结论	5
7.1	主要成果	5
7.2	未来工作	6

1 引言

1.1 背景

在现代计算环境中，用户通常需要管理数以百万计的文件。传统的文件搜索方法（如基于文件名的精确匹配）往往无法满足快速、模糊搜索的需求。本项目旨在构建一个基于向量相似度的文件搜索系统，能够快速定位用户所需的文件。

1.2 目标

- 索引大规模文件系统（支持千万级文件）
- 提供毫秒级搜索响应
- 支持模糊搜索和语义相似度匹配
- 提供友好的 Web 界面和 API 接口

2 系统架构

2.1 整体架构

系统采用三层架构设计：

1. 数据层：基于 RocksDB 的向量数据库，存储文件向量和元数据
2. 服务层：Python Flask Web 服务，提供搜索 API
3. 表示层：Web 界面，提供用户交互

2.2 核心组件

2.2.1 CoreNN 向量数据库

CoreNN 是使用 Rust 实现的高性能向量数据库，采用 HNSW (Hierarchical Navigable Small World) 算法进行近似最近邻搜索。

主要特性：

- 支持 128 维浮点向量
- 基于 RocksDB 的持久化存储
- HNSW 索引结构，提供 $O(\log n)$ 搜索复杂度
- 支持批量插入和查询

2.2.2 文件索引器

文件索引器负责扫描文件系统并生成向量表示。

向量特征构成 (128 维):

- 文件名特征 (32 维): 基于 MD5 哈希
- 扩展名特征 (16 维): 文件类型标识
- 目录路径特征 (32 维): 文件位置信息
- 文件大小特征 (16 维): 归一化的文件大小
- 修改时间特征 (16 维): 时间戳信息
- MIME 类型特征 (16 维): 文件类型分类

3 实现细节

3.1 索引构建流程

1. 文件扫描: 使用 `os.walk()` 遍历文件系统
2. 特征提取: 为每个文件生成 128 维向量
3. 向量归一化: 确保向量模长为 1
4. 批量插入: 将向量写入 CoreNN 数据库
5. 元数据保存: 将文件信息保存为 JSON

3.2 搜索算法

搜索过程分为以下步骤:

1. 将查询字符串转换为 128 维向量
2. 使用 HNSW 算法在向量空间中查找最近邻
3. 计算余弦相似度:

$$\text{similarity} = \frac{\mathbf{q} \cdot \mathbf{v}}{|\mathbf{q}| \cdot |\mathbf{v}|} \quad (1)$$

4. 返回 Top-K 个最相似的文件

4 性能分析

4.1 索引构建性能

4.2 搜索性能

4.3 内存使用

- 索引构建峰值内存: 25.8 GB

指标	数值
索引文件总数	9,431,950
总耗时	2 小时 10 分钟
处理速度	72,400 文件/分钟
平均每文件	0.83 毫秒
向量数据库大小	11 GB
元数据大小	4.2 GB
总索引大小	15.2 GB

Table 1: 索引构建性能统计

操作	响应时间
单次查询	< 100 ms
Top-10 结果	50-80 ms
Top-50 结果	80-150 ms

Table 2: 搜索响应时间

- 搜索服务运行内存: 20.3 GB
- 向量数据库加载时间: 约 30 秒

5 系统使用

5.1 启动服务

```

1 # 激活虚拟环境
2 source venv/bin/activate
3
4 # 启动服务器
5 python file_search_server.py

```

Listing 1: 启动搜索服务器

5.2 API 使用示例

```

1 # 搜索文件
2 curl -X POST http://localhost:5000/search \
3   -H "Content-Type: application/json" \
4   -d '{"query": "config.json", "limit": 10}'
5
6 # 查看统计信息
7 curl http://localhost:5000/stats

```

Listing 2: 搜索 API 调用

6 问题与优化

6.1 当前问题

6.1.1 搜索结果准确性

当前搜索算法基于简单的 MD5 哈希特征，可能导致以下问题：

- 语义相似但名称不同的文件无法关联
- 哈希冲突可能影响搜索准确性
- 缺乏对文件内容的理解

6.2 优化方案

6.2.1 改进向量表示

1. 使用 **N-gram** 特征：将文件名分解为字符级 N-gram，提高模糊匹配能力
2. **TF-IDF** 权重：对常见词降权，提高区分度
3. 语义嵌入：使用预训练模型（如 BERT）生成语义向量

6.2.2 优化搜索算法

1. 多阶段检索：先粗筛选，再精排序
2. 查询扩展：自动添加同义词和相关词
3. 个性化排序：根据用户历史调整结果

7 结论

本项目成功实现了一个高性能的大规模文件索引与搜索系统，能够在毫秒级时间内从近千万个文件中检索目标文件。系统采用向量相似度搜索技术，提供了比传统文件搜索更灵活的检索方式。

7.1 主要成果

- 成功索引 9,431,950 个文件
- 实现毫秒级搜索响应（< 100ms）
- 提供 Web 界面和 API 接口
- 系统稳定运行，内存占用合理

7.2 未来工作

- 改进向量表示方法，提高搜索准确性
- 支持增量索引更新
- 添加文件内容索引功能
- 优化内存使用，降低系统资源消耗

参考文献

1. Malkov, Y. A., & Yashunin, D. A. (2018). Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE transactions on pattern analysis and machine intelligence*, 42(4), 824-836.
2. RocksDB: A Persistent Key-Value Store for Fast Storage Environments. Facebook, Inc.
3. Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*.